module and package in py all concept and code

Here is **Module and Package in Python — All Concepts + Clear Examples + Code** Explained in a simple, structured, beginner-to-advance format.

✅ MODULES & PACKAGES IN PYTHON (Complete Notes + Code)

📌 1. What is a Module in Python?

A **module** = a single **.py file** that contains code (functions, classes, variables).

✔️ Why use modules?

- Code reusability
- Better organization
- Avoid huge files
- Encapsulation

🎯 2. Creating a Module

📁 mymodule.py

python Copy code

```python
# This is a module

pi = 3.14

def add(a, b):
    return a + b

def greet(name):
    return f"Hello, {name}!"
```

🎯 3. Importing a Module

1️⃣ import module

python Copy code

```python
import mymodule

print(mymodule.pi)
print(mymodule.add(3, 4))
```

2️⃣ import module as alias

python Copy code

```python
import mymodule as m

print(m.add(10, 5))
```

3️⃣ from module import function

python Copy code

```python
from mymodule import add, pi

print(add(2, 3))
```

4️⃣ from module import * (NOT recommended)

python Copy code

```python
from mymodule import *

print(greet("Aamir"))
```

📌 4. Built-in Modules

Examples:

python Copy code

```python
import math
import random
import datetime
import os
import sys
```

📌 5. dir() function

Shows all variables, functions inside module.

python Copy code

```python
import math
print(dir(math))
```

📌 6. name == "main"

Used to check if file is:

- Run directly
- Imported as module

mymodule.py

python Copy code

```python
def hello():
    print("Hello Aamir")

if __name__ == "__main__":
    print("File executed directly")
else:
    print("File imported")
```

📌 7. Reloading a Module

When module changes, reload:

python Copy code

```
import importlib
import mymodule

importlib.reload(mymodule)
```

🟦 🟪 PACKAGES IN PYTHON (Full Concepts)

📌 1. What is a Package?

A **package** = a folder that contains multiple Python modules.

A package contains:

- A folder
- One or more `.py` files
- `__init__.py` file

📁 Package Structure Example

csharp Copy code

```
myproject/
    mathpkg/
        __init__.py
        add.py
        subtract.py
        multiply.py
    main.py
```

📌 2. Code inside package files

📁 mathpkg/add.py

python Copy code

```
def add(a, b):
    return a + b
```

📁 mathpkg/subtract.py

python Copy code

```python
def sub(a, b):
    return a - b
```

📁 mathpkg/multiply.py

python Copy code

```python
def mul(a, b):
    return a * b
```

📌 3. init.py

`__init__.py` makes the folder a **package**.

📁 mathpkg/init.py

python Copy code

```python
from .add import add
from .subtract import sub
from .multiply import mul
```

📌 4. Using the Package

📁 main.py

python Copy code

```python
from mathpkg import add, sub, mul

print(add(3, 4))
```

```
print(sub(10, 5))
print(mul(2, 6))
```

📌 5. Importing module inside package

python Copy code

```python
import mathpkg.add
print(mathpkg.add.add(2, 3))
```

📌 6. Nested Packages (Sub-packages)

arduino Copy code

```arduino
mypkg/
    __init__.py
    geometry/
        __init__.py
        circle.py
        square.py
```

circle.py

python Copy code

```python
def area(r):
    return 3.14 * r * r
```

Using subpackage:

python Copy code

```python
from mypkg.geometry.circle import area
print(area(5))
```

📌 7. Absolute vs Relative Imports

✔️ Absolute Import

python Copy code

```python
from mathpkg.add import add
```

✔️ Relative Import

Inside same package:

python Copy code

```python
from .add import add
from .subtract import sub
```

📌 8. Installing Packages (from pip)

bash Copy code

```bash
pip install numpy
pip install pandas
```

Using them:

python Copy code

```python
import numpy as np
import pandas as pd
```

📌 9. Publishing a Package (Basic Steps)

1. Create package folder
2. Add `setup.py`
3. Build:

bash Copy code

```bash
python setup.py sdist bdist_wheel
```

4. Upload to PyPI:

bash

```
twine upload dist/*
```

🧠 INTERVIEW QUESTIONS (Very Important)

✅ 1. What is the difference between module and package?

Module	Package
A single `.py` file	A folder containing multiple modules
Smaller component	Larger collection
No `__init__.py` needed	`__init__.py` required

✅ 2. Why do we use `__init__.py` ?

- Converts folder into package
- Initializes package
- Controls imports

✅ 3. Types of imports?

- Absolute
- Relative
- Selected imports
- - import (not recommended)

✅ 4. Can a package contain another package?

Yes → **Sub-packages**

✅ 5. What is `if __name__ == "__main__"` used for?

- To execute code only when file is run directly
- Prevent execution when imported

✔ Want PDF Notes?

I can generate a **clean PDF (with tables + examples + interview questions + diagrams)**.

Just say **"make PDF"**.

Exception handling describe all concept

Below is **Exception Handling in Python — ALL CONCEPTS + Code Examples**
Crystal-clear explanation from **basic → advanced**, including **errors, try-except, custom exception, raising errors, chaining, finally, else, best practices, interview questions**.

🟥 1. What is an Exception?

An **exception** is an error that occurs during program execution.

Examples:

- ZeroDivisionError
- TypeError
- ValueError
- FileNotFoundError
- IndexError

When an exception occurs → program stops.

Exception handling ensures the program does NOT crash.

🟦 2. Types of Errors

✔ Syntax Error

Error in code structure → detected before execution.

python Copy code

```
print("Hello"
```

✔️ Runtime Error (Exception)

Error during execution.

python Copy code

```python
x = 10 / 0
```

🟪 3. Basic try-except

python Copy code

```python
try:
    a = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
```

🟩 4. Multiple Except Blocks

python Copy code

```python
try:
    num = int("abc")
except ValueError:
    print("Invalid number")
except TypeError:
    print("Type error")
```

🟫 5. Single except for multiple exceptions

python Copy code

```python
try:
    a = 10 / 0
```

```
except (ZeroDivisionError, ValueError):
    print("Error occurred")
```

🟥 6. Catch ALL exceptions

python Copy code

```python
try:
    x = int("hello")
except Exception as e:
    print("Error:", e)
```

Never use only `except:` (bad practice).

🟨 7. else block

`else` executes **ONLY when no exception occurs**.

python Copy code

```python
try:
    x = int(input("Enter number: "))
except ValueError:
    print("Invalid input")
else:
    print("Valid number:", x)
```

🟦 8. finally block

`finally` executes **no matter what**

→ for cleanup (closing file, DB connection)

python Copy code

```python
try:
    file = open("data.txt")
    print(file.read())
except FileNotFoundError:
    print("File not found")
```

```
finally:
    print("Closing file...")
    file.close()
```

🟧 9. try with all blocks together

python Copy code

```python
try:
    x = 5 / 1
except ZeroDivisionError:
    print("Error")
else:
    print("No error")
finally:
    print("Always runs")
```

🟪 10. Raising (Throwing) Exception

Use `raise` to generate your own error.

python Copy code

```python
age = -5
if age < 0:
    raise ValueError("Age cannot be negative")
```

🟥 11. Custom Exceptions

Create your own exception class:

python Copy code

```python
class InsufficientBalanceError(Exception):
    pass

balance = 100
withdraw = 150
```

```
if withdraw > balance:
    raise InsufficientBalanceError("Not enough balance")
```

🟩 12. Try-except inside functions

python Copy code

```python
def divide(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        return "Division by zero not allowed"

print(divide(10, 0))
```

🟫 13. Exception Chaining (`raise from`)

Used to show original cause of error.

python Copy code

```python
try:
    num = int("abc")
except ValueError as e:
    raise TypeError("Type conversion failed") from e
```

🟥 14. Using `assert` Keyword

`assert` raises `AssertionError` if condition fails.

python Copy code

```python
x = -1
assert x >= 0, "x must be positive"
```

🟨 15. Logging instead of print

Better to log errors for debugging:

python Copy code

```python
import logging

try:
    a = 10 / 0
except ZeroDivisionError as e:
    logging.error("Zero division error: %s", e)
```

🟦 16. Nested try-except

python Copy code

```python
try:
    try:
        x = int("abc")
    except ValueError:
        print("Inner value error")
except Exception:
    print("Outer exception")
```

🟪 17. Using pass (ignore exception)

python Copy code

```python
try:
    x = int("abc")
except:
    pass
```

Not recommended — hides bugs.

🟥 18. File Handling Exceptions

python Copy code

```
try:
    f = open("abc.txt")
except FileNotFoundError:
    print("File does not exist")
```

🟩 19. Exception Hierarchy (Important)

php Copy code

```
BaseException
    ├── SystemExit
    ├── KeyboardInterrupt
    └── Exception
        ├── ArithmeticError
        │    └── ZeroDivisionError
        ├── LookupError
        │    ├── IndexError
        │    └── KeyError
        ├── ValueError
        ├── TypeError
        ├── FileNotFoundError
        └── ...
```

🧠 INTERVIEW QUESTIONS (Very Important)

✔️ 1. What is exception handling?

Mechanism to handle runtime errors without crashing program.

✔️ 2. Difference between errors and exceptions?

Errors	Exceptions
Mistake in code structure	Occur at runtime
Cannot be handled	Can be handled using try-except
Example: SyntaxError	Example: ZeroDivisionError

✔ 3. Purpose of finally block?

Runs always → cleanup code.

✔ 4. What is custom exception?

User-defined exception using class.

✔ 5. What is `raise`?

Used to throw an exception manually.

✔ 6. What is the order of except blocks?

More specific exceptions should come first.

✔ 7. What is `assert`?

Debugging tool → raises AssertionError when condition fails.

✔ 8. Can finally block skip execution?

Only if:

- `os._exit()`
- Program crash
- Power failure

Otherwise → always executes.

✔ Want a PDF with Table + Diagrams + Code + Interview Questions?

Just say **"make PDF"**.

Table + Diagrams + Code + Interview Questions?

Great! I can generate a **full PDF** for you containing:

- ✔ Exception Handling **all concepts**
- ✔ Clean **tables**
- ✔ Visual **diagrams**
- ✔ All **code examples**
- ✔ 25+ **interview questions & answers**
- ✔ Proper headings, formatting, and layout

Before I generate the PDF, tell me **which style you prefer**:

📝 Choose PDF Style

1. **Simple Notes Style**
   - Clear headings
   - Bullet points
   - Code blocks
2. **Textbook/Guide Style**
   - Section-wise explanation
   - Diagrams
   - Tables
   - Code + Output
   - Best for studying
3. **Interview-Focused Style**
   - Concepts
   - Quick tables
   - Direct Q/A
   - Code examples only where needed
4. **Premium Notes Style (Recommended)**
   - Like coaching institute notes
   - With diagrams
   - Big tables
   - All examples
   - Q/A
   - Clean formatting

⭐ Please reply with a number:

1, 2, 3, or 4

Once you choose, I will generate the **professional PDF** for download.

describe all

I want to confirm what you mean by **"describe all."**
Right now, there are **two possible meanings**, so please choose the correct one:

✅ Option A — You want me to *describe all PDF styles* (1,2,3,4).

Meaning:
You want an explanation of:

- Simple Notes Style
- Textbook Style
- Interview Style
- Premium Notes Style

→ **I will NOT create the PDF yet.**
I will just explain what each style looks like.

✅ Option B — You want me to *include everything* in the PDF

Meaning:
You want a PDF that contains:

✔ Tables
✔ Diagrams
✔ All code
✔ All interview questions
✔ All concepts fully detailed

→ **I will create the final PDF with everything.**

❓ Please tell me which one you mean: A or B?

Just reply with **A** or **B**.

Table + Diagrams + Code + Interview Questions? explain with all ans

Thought for 4 seconds

Great — you want a complete, self-contained explanation that includes **Tables**, **Diagrams**, **Code examples**, and **Interview Questions + Answers** covering **Exception Handling in Python**. Below I've packed everything into one tidy guide you can read straight away or copy into a PDF later.

Exception Handling — Full Guide (Tables • Diagrams • Code • Q/A)

---

1) Quick summary table — common exceptions

Exception Name	When it occurs	Example
`ZeroDivisionError`	Division or modulo by zero	`1/0`
`ValueError`	Wrong value type/format for operation	`int('x')`
`TypeError`	Operation on wrong type	`'a' + 1`
`IndexError`	Sequence index out of range	`lst[10]`
`KeyError`	Missing dict key	`d['missing']`
`FileNotFoundError`	Opening non-existent file	`open('no.txt')`

Exception Name	When it occurs	Example
`IOError` / `OSError`	I/O or OS-level error	file system issues
`AttributeError`	Access invalid attribute	`obj.missing()`
`ImportError` / `ModuleNotFoundError`	import fails	`import nonexist`
`AssertionError`	`assert` fails	`assert x>0`
`StopIteration`	Iterator exhausted	`next(iter([]))`
`KeyboardInterrupt`	User interrupts (Ctrl+C)	—
`Exception`	Base for most user-level exceptions	catch-all

---

2) Control-flow diagram (ASCII)

(Shows order: `try` → exception? → `except` → `else` → `finally`)

powershell Copy code

```
Start
  |
  ▼
 try:
  ├── if no exception ──► else block (optional)
  |                        |
  |                        ▼
  |                     finally
  |                        |
  |                        ▼
  └── if exception ──► matching except block(s)
                          |
                          ▼
                       (maybe re-raise)
                          |
                          ▼
                       finally
                          |
```

```
▼
End
```

---

3) Core concepts with runnable code examples

3.1 Basic `try` / `except`

python Copy code

```python
try:
    result = 10 / 2
    print("Result:", result)
except ZeroDivisionError:
    print("Can't divide by zero")
```

3.2 Catch multiple specific exceptions

python Copy code

```python
try:
    x = int(input("Enter an integer: "))
    y = 10 // x
except ValueError:
    print("Not an integer.")
except ZeroDivisionError:
    print("Can't divide by zero.")
```

3.3 One `except` for multiple exception types

python Copy code

```python
try:
    # some code
    pass
except (TypeError, ValueError) as e:
    print("Type or value problem:", e)
```

3.4 Catch-all with logging (use sparingly)

python Copy code

```python
import logging

try:
    do_something()
except Exception as e:
    logging.exception("Unexpected error")
    # handle or re-raise
```

3.5 `else` block (runs if no exception)

python Copy code

```python
try:
    val = int("42")
except ValueError:
    print("Bad conversion")
else:
    print("Converted value:", val)  # runs because no exception
```

3.6 `finally` (always runs — cleanup)

python Copy code

```python
f = None
try:
    f = open("data.txt")
    data = f.read()
except FileNotFoundError:
    print("Missing file")
finally:
    if f:
        f.close()   # guaranteed cleanup
```

3.7 Raising an exception manually

python Copy code

```python
def set_age(age):
    if age < 0:
        raise ValueError("age must be non-negative")
    return age
```

3.8 Custom exception classes

python Copy code

```python
class InsufficientFunds(Exception):
    """Raised when withdrawal exceeds balance"""
    pass

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientFunds(f"Need {amount - balance} more")
```

3.9 Exception chaining (`raise ... from ...`)

python Copy code

```python
try:
    x = int("abc")
except ValueError as e:
    raise RuntimeError("Parsing failed in config") from e
```

3.10 Re-raising an exception after handling or logging

python Copy code

```python
try:
    risky()
except Exception as e:
    log(e)
    raise  # re-raises same exception
```

3.11 Context manager and exceptions (`with`)

`with` simplifies cleanup (calls `__exit__` even on exceptions).

python Copy code

```python
with open("file.txt") as f:
    data = f.read()  # file auto-closed even on error
```

3.12 Using `assert` (debug checks)

python Copy code

```python
def sqrt(x):
    assert x >= 0, "x must be non-negative"
    return x**0.5
```

Note: `assert` can be disabled with `-O` flag; use for debugging, not regular error handling.

3.13 Best-practice pattern for resource cleanup

Use context managers or `try/finally`. Prefer `with` when available.

---

4) Advanced topics & patterns

4.1 Exception hierarchy and ordering

Always catch **specific** exceptions before general ones:

python Copy code

```python
try:
    ...
except FileNotFoundError:
    ...
except Exception:  # generic — last
    ...
```

4.2 Translating exceptions for API boundaries

Wrap low-level exceptions into higher-level domain exceptions (with chaining).

4.3 Validation vs Exceptions

Use exceptions for exceptional conditions — not ordinary control flow.

4.4 Clean error messages

Include useful context: values, operation attempted, suggestions for fix.

4.5 When to log vs raise

- Library code: raise exceptions (let caller decide).
- Top-level app code: catch, log, present friendly message or retry.

4.6 Avoid broad `except:` and swallowing exceptions

`except Exception as e:` is preferable to bare `except:` because it avoids catching `SystemExit`, `KeyboardInterrupt`.

---

5) Tables — `try/except/else/finally` behavior

Situation	`except` runs?	`else` runs?	`finally` runs?
No exception	No	Yes	Yes
Caught exception	Yes	No	Yes
Uncaught exception	No (unless same-level handler)	No	Yes (usually)
`sys.exit()` called	Depends	No	No (may prevent finally)

---

6) Short cheatsheet — what to `catch` when

- `FileNotFoundError` — missing file open/read
- `ValueError` — bad input conversion
- `TypeError` — wrong type usage
- `KeyError` — missing dict key (use `dict.get()` to avoid)
- `IndexError` — list index out of range (check length)
- `OSError` / `IOError` — file/OS related operations
- `Exception` — fallback, use with logging + re-raise

---

7) Common pitfalls (and fixes)

- **Swallowing exceptions**: `except: pass` hides bugs → log or re-raise.
- **Catching too broadly**: Mask serious errors; be specific.
- **Relying on `assert` for input validation**: Use explicit checks and raise proper exceptions.
- **Not using context managers**: Resource leaks — prefer `with`.
- **Order of except blocks reversed**: `except Exception` before `ValueError` will make `ValueError` unreachable.

---

8) Diagrams: Exception object flow (textual)

When an exception occurs:

1. Python creates an exception object with type, value, traceback.
2. It searches up the call stack for matching `except`.
3. If found — that handler executes; else program terminates and prints traceback.
4. `finally` blocks execute on the way out (if possible).

---

9) Interview Questions (25+) — with concise model answers

1. **Q:** What is an exception in Python?
   A: An exception is a runtime error signaled by an exception object. It interrupts normal flow and can be handled using `try/except`.
2. **Q:** Difference between syntax error and exception?
   A: Syntax error detected at parse time (program won't run). Exception occurs at runtime while executing valid code.
3. **Q:** Purpose of `try`, `except`, `else`, `finally`?
   A: `try` wraps code that might fail; `except` handles errors; `else` executes when no exception; `finally` always executes for cleanup.
4. **Q:** Why put `finally`?
   A: To guarantee cleanup (closing files/connections) regardless of success or failure.
5. **Q:** How to raise an exception?
   A: Use `raise ExceptionType("message")`.
6. **Q:** How to define a custom exception?
   A: Subclass `Exception`, e.g. `class MyError(Exception): pass`.
7. **Q:** What is `raise ... from ...`?
   A: It chains exceptions, preserving the original cause (`__cause__`) for debugging.
8. **Q:** When should you use `assert`?
   A: For internal consistency checks during development; not for user input validation.
9. **Q:** Difference between `except Exception:` and bare `except:`?
   A: `except Exception` catches most user-level exceptions; bare `except` catches everything including `SystemExit` and `KeyboardInterrupt` (not recommended).
10. **Q:** How to re-raise an exception after handling?
    A: Use `raise` with no arguments inside an `except` block.
11. **Q:** What is exception chaining? How is it useful?
    A: Chaining links lower-level cause to higher-level exception to preserve original traceback for debugging.

12. **Q:** Best practice to handle file operations?
    A: Use `with open(...) as f:` which ensures file is closed even if an error happens.
13. **Q:** How to catch multiple exceptions in a single `except`?
    A: `except (TypeError, ValueError) as e:`.
14. **Q:** Should library functions catch exceptions?
    A: Libraries should raise meaningful exceptions and document them; they may catch low-level exceptions to wrap them into domain-specific ones.
15. **Q:** How to get the exception message and traceback?
    A: Use `except Exception as e:` to get message; `traceback` module or `logging.exception()` for full traceback.
16. **Q:** What is `sys.exc_info()`?
    A: Returns a tuple `(type, value, traceback)` for the current exception being handled.
17. **Q:** When might `finally` not execute?
    A: If the process is killed, `os._exit()` is called, or system-level crash/power loss.
18. **Q:** Difference between `OSError` and `IOError`?
    A: In Python 3, `IOError` was merged into `OSError`. Use `OSError` for general OS-related errors.
19. **Q:** How to avoid catching exception but still be safe?
    A: Catch specific exceptions, log context, and re-raise if necessary.
20. **Q:** What is the advantage of raising custom exceptions?
    A: They provide clearer error semantics and allow callers to handle domain-specific errors.
21. **Q:** How to test code that raises exceptions?
    A: Use `pytest`'s `with pytest.raises(ExpectedError):` or `unittest`'s `assertRaises`.
22. **Q:** Can exceptions be used for flow control?
    A: Technically yes, but it's discouraged — exceptions are for exceptional cases and incur performance cost.
23. **Q:** How to print a friendly error for users but still log tracebacks?
    A: Catch exceptions at top level, log full exception with `logging.exception()`, and show a sanitized message to user.
24. **Q:** Is it okay to use broad exception handling in web request handlers?
    A: Top-level handlers may catch broad exceptions to return 500 responses, but deeper code should use specific handling.
25. **Q:** How do context managers handle exceptions?
    A: The `__exit__` method receives exception type, value, and traceback; if it returns

`True` , it suppresses the exception.

26. **Q:** How to get the traceback as a string?

    A: Use `import traceback; traceback.format_exc()` .

27. **Q:** What's the difference between `Exception` and `BaseException` ?

    A: `BaseException` is the root; `Exception` is for typical errors. `SystemExit` , `KeyboardInterrupt` inherit from `BaseException` but not `Exception` .

28. **Q:** Should you catch `KeyboardInterrupt` ?

    A: Only if you need to do special cleanup; otherwise let it propagate so user can interrupt.

---

10) Example: Full sample module demonstrating concepts

python Copy code

```python
# exceptions_demo.py
import logging, traceback

logging.basicConfig(level=logging.INFO)

class AppError(Exception):
    pass

def parse_int(s):
    try:
        return int(s)
    except ValueError as e:
        # wrap low-level error into app-level error
        raise AppError(f"Invalid integer: {s}") from e

def read_file(path):
    try:
        with open(path) as f:
            return f.read()
    except FileNotFoundError:
        logging.error("File not found: %s", path)
        raise

def main():
    try:
        x = parse_int("abc")
        content = read_file("nofile.txt")
```

```
    except AppError as ae:
        logging.error("Application error: %s", ae)
    except Exception:
        logging.exception("Unhandled exception")
    finally:
        print("Shutdown tasks run here")

if __name__ == "__main__":
    main()
```

---

11) Best practices checklist (copy-paste)

- ☐ Catch specific exceptions, not broad ones (except at top-level boundary).
- ☐ Always clean up resources — prefer `with`.
- ☐ Log exceptions with full traceback (`logging.exception`).
- ☐ Use custom exceptions for domain-specific errors.
- ☐ Avoid using exceptions for normal control flow.
- ☐ Provide helpful messages and include contextual data.
- ☐ Re-raise after partial handling if the caller needs to know.
- ☐ Document which exceptions your functions may raise.

---

12) Want this as a PDF (formatted with diagrams & code)?

If you'd like, I can generate a **professional PDF** with the above content, formatted headings, nice diagrams, and code blocks. Reply **"Make PDF"** and tell me:

- Filename (optional), e.g. **exception_handling_notes.pdf**
- Which style you prefer (Simple / Textbook / Interview / Premium) — if you want a choice.

If you want, I can also:

- Add **more examples** for specific use-cases (web apps, DB transactions, async code).
- Show **unit tests** that assert exception raising.
- Produce **slides** or a compact cheat-sheet.

Which of those next steps would you like?

file handling describe all concept

Here is **File Handling – All Concepts (Beginner to Advanced)** explained in the clearest and most complete way.

⭐ FILE HANDLING — COMPLETE NOTES (Python + General Concepts)

File handling allows programs to **create**, **read**, **write**, **update**, and **delete** files on the system.

🔶 1. Why File Handling?

We need file handling when:

- You want to **store data permanently**
- You want to **read existing files** (CSV, text, logs)
- You want to **write program output** to a file
- You want to **append new data** continuously (logs, reports)
- You want to **create structured data** (JSON, CSV)
- You want to **handle big data** that cannot fit in memory

🔶 2. File Types

Text Files

- `.txt`
- `.csv`
- `.json`
- `.xml`
- `.html`

Binary Files

- `.jpg`, `.png`
- `.mp4`

- `.pdf`
- `.exe`
- `.dat`

  Binary files store raw bytes, not human-readable text.

🔶 3. Basic File Operations

Operation	Python Function	Mode
Reading	`file.read()`	`r`
Writing	`file.write()`	`w`
Appending	`file.write()`	`a`
Reading/Writing	—	`r+`, `w+`, `a+`
Binary Read	`read()`	`rb`
Binary Write	`write()`	`wb`

🔶 4. File Opening Modes

Mode	Meaning	If File Not Found	Overwrite?
`r`	Read	❌ Error	No
`w`	Write	✅ Created	✅ Yes
`a`	Append	✅ Created	No
`r+`	Read + Write	❌ Error	No
`w+`	Read + Write	✅ Created	✅ Yes
`a+`	Read + Append	✅ Created	No
`rb` / `wb`	Binary Read/Write	—	—

---

5. Using `open()` function

python Copy code

```python
file = open("data.txt", "r")
content = file.read()
file.close()
```

---

6. Best Practice: `with` Statement (Auto Close)

python Copy code

```python
with open("data.txt", "r") as f:
    print(f.read())
```

Benefits:

- Auto close
- No memory leak
- No file corruption

---

7. Reading from a File

1. Read entire file

python Copy code

```python
f.read()
```

2. Read line by line

python Copy code

```python
for line in f:
    print(line)
```

3. Read one line

python Copy code

```
f.readline()
```

4. Read all lines as list

python Copy code

```python
f.readlines()
```

🔶 8. Writing to a File

This overwrites old content.

python Copy code

```python
with open("data.txt", "w") as f:
    f.write("Hello Aamir")
```

🔶 9. Appending to a File

Adds new content at end.

python Copy code

```python
with open("data.txt", "a") as f:
    f.write("\nNew Line Added")
```

🔶 10. File Pointer (`seek` & `tell`)

tell() → Current cursor position

python Copy code

```python
pos = f.tell()
print(pos)
```

seek() → Move cursor

python Copy code

```
f.seek(0)     # Move to beginning
f.seek(5)     # Move to 5th byte
```

---

11. Checking File Exists

python Copy code

```python
import os

os.path.exists("data.txt")  # True or False
```

---

12. Deleting a File

python Copy code

```python
import os
os.remove("data.txt")
```

---

13. Working With Directories

python Copy code

```python
os.mkdir("myfolder")          # create folder
os.listdir()                  # list files
os.rmdir("myfolder")          # remove empty folder
```

---

14. Exception Handling in Files

python Copy code

```python
try:
    with open("data.txt","r") as f:
        print(f.read())
```

```
except FileNotFoundError:
    print("File not found")
```

---

15. File Handling With CSV

Reading CSV

python Copy code

```python
import csv

with open("data.csv") as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

Writing CSV

python Copy code

```python
with open("data.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["Name","Age"])
    writer.writerow(["Aamir",22])
```

---

16. File Handling With JSON

Write JSON

python Copy code

```python
import json

data = {"name":"Aamir","age":22}

with open("data.json","w") as f:
    json.dump(data,f,indent=4)
```

Read JSON

python Copy code

```python
with open("data.json") as f:
    d = json.load(f)
```

🔶 17. File Handling With Binary Files

Read image (binary)

python Copy code

```python
with open("image.png","rb") as f:
    content = f.read()
```

Write binary

python Copy code

```python
with open("copy.png","wb") as f:
    f.write(content)
```

🔶 18. File Handling in Java (Short + Clear)

Read file (Java)

java Copy code

```java
FileReader fr = new FileReader("data.txt");
BufferedReader br = new BufferedReader(fr);

String line;
while((line = br.readLine()) != null){
    System.out.println(line);
}

br.close();
```

Write file (Java)

java Copy code

```
FileWriter fw = new FileWriter("data.txt");
fw.write("Hello World");
fw.close();
```

🔶 19. File Handling – Interview Questions (Important)

✔ What is file pointer?

Position from where next read/write happens.

✔ Difference between text & binary file?

Text → characters

Binary → raw bytes

✔ Difference between `w` and `a` mode?

`w` → overwrite

`a` → append

✔ Why use `with` statement?

Auto-closes file safely.

✔ What is buffering?

Temporary storage while reading/writing.

✔ What happens if `open("file.txt","r")` and file doesn't exist?

Throws `FileNotFoundError`.

🔶 20. All Core Concepts Summary

Concept	Covered
File Types	✔
Modes	✔
Read/Write/Append	✔
File Pointer	✔

Concept	Covered